Infraestructura Kubernetes para desarrollo y producción

Práctica integradora individual de Infraestructura de Servicios

Cada estudiante diseñará, implementará y documentará dos ambientes Kubernetes para una aplicación multicapa de creación propia. El resultado será una infraestructura reproducible, con seguridad, persistencia, observabilidad, balanceo y escalado horizontal verificables.

Producto final

Una aplicación funcional disponible por HTTPS en desarrollo y en producción de laboratorio, dos clústeres independientes y un expediente técnico que permita a otra persona reconstruirlos desde equipos limpios. El alumno entregará código, infraestructura declarativa, manuales, resultados de pruebas y evidencias propias.

Datos de identificación

Alumno _____

Nombre del proyecto _____

Repositorio y versión de entrega _____

Fecha de inicio y fecha de entrega _____

Duración sugerida y modalidad

Individual. dos semanas. Cada estudiante será responsable de aplicación, datos, redes, seguridad y operación. Se permiten bibliotecas y herramientas existentes con atribución; el diseño, la integración y la defensa serán personales.

Alcance de producción

El ambiente productivo es una simulación académica con controles de operación. No se usarán datos personales reales. La práctica exige disponibilidad de la capa de aplicación; la alta disponibilidad completa del plano de control y de las bases de datos se diseña y se identifica como ampliación cuando no se implemente.

1 Competencias y elección del proyecto

Al finalizar, el estudiante podrá justificar una arquitectura multicapa, operar recursos Kubernetes, segmentar comunicaciones, proteger datos y demostrar mediante mediciones la respuesta ante carga y fallos.

Proyecto personal distinto

Antes de implementar, presentar una ficha de dos páginas con problema, usuarios, alcance, reglas de negocio, módulos, tecnologías y uso de cada base de datos. El docente registrará los proyectos para evitar duplicados. Cambiar únicamente el nombre, colores o catálogo no constituye un proyecto distinto.

- Ejemplos de dominios: préstamo de instrumentos musicales, reservas de canchas, seguimiento de reparación de bicicletas, rescate y adopción animal, administración de huertos o control de exposiciones artísticas.
- La diferenciación debe incluir al menos tres reglas de negocio propias, un flujo con cambio de estado y una justificación del valor de la base no relacional.

Investigación técnica obligatoria

Explicar con fuentes oficiales y ejemplos del proyecto: plano de control y nodos; Pod y Deployment; Service y descubrimiento DNS; CNI y NetworkPolicy; CSI y almacenamiento; StatefulSet; configuración y secretos; RBAC; sondas; HPA; distribución de réplicas; actualizaciones y recuperación.

Comparar dos alternativas de distribución Kubernetes y dos opciones de entrada de tráfico. Justificar la seleccionada por recursos, soporte, seguridad y facilidad de reproducción. No se exige usar microservicios: un backend modular puede representar correctamente la capa de negocio.

Preguntas para la defensa

- ¿Qué ocurre desde que el usuario abre la URL hasta que se consulta cada base de datos?
- ¿Qué diferencia existe entre reiniciar un contenedor, recrear un Pod, escalar réplicas y añadir nodos?
- ¿Qué seguirá funcionando si falla un Pod, un worker, el control plane o el almacenamiento?
- ¿Cómo se demuestra que una conexión bloqueada falla por la política y no por un servicio caído?

2 Requisitos funcionales de la aplicación

Elemento	Mínimo verificable
Presentación	Interfaz navegable y adaptable, formularios, mensajes de error y confirmación de operaciones.
Negocio	Backend con API y reglas de dominio. No permitir acceso del navegador directamente a las bases de datos.
CRUD	Tres entidades de negocio relacionadas con altas, consultas, modificaciones y bajas lógicas o físicas justificadas.
Usuarios	Autenticación y al menos dos roles con permisos distintos. Verificar autorización en servidor.
Validación	Tipos, campos obligatorios, rangos, unicidad y relaciones. Validar en cliente y servidor.
Consultas	Búsqueda, paginación y un reporte útil al proyecto.
Flujo propio	Proceso con estados y transiciones válidas, por ejemplo solicitud, aprobación y cierre.
Datos	Semilla reproducible de al menos 100 registros de negocio y usuarios de prueba sin datos reales.

Base relacional

Usar un SGBD relacional con llaves primarias y foráneas, restricciones, índices y transacciones para las operaciones que lo requieran. Entregar diagrama entidad relación, diccionario de datos, migraciones y consultas representativas. Justificar la estrategia de conexiones para varias réplicas del backend.

Base no relacional

Usar una base no relacional para una función real visible en la aplicación: bitácora consultable, documentos flexibles, historial de cambios o contenido del dominio. Entregar modelo, validaciones, índices y consultas. Una base instalada sin uso no cumple; una caché por sí sola no sustituye este requisito.

Definir cuál sistema es fuente de verdad y qué sucede si una escritura en la segunda base falla. No suponer atomicidad entre dos motores. Implementar reintento idempotente, patrón outbox o una estrategia equivalente, y demostrar recuperación sin duplicados.

Pruebas funcionales

Incluir casos válidos, datos inválidos, duplicados, usuario sin permisos, sesión expirada y caída temporal de una dependencia. Las validaciones deben impedir cambios indebidos aunque se evite la interfaz gráfica.

3 Diseño de los dos ambientes

Se requieren dos clústeres independientes con contextos, credenciales, configuración, datos y volúmenes separados. Dos namespaces dentro de un mismo clúster no sustituyen los dos ambientes. Se permite alojarlos en el mismo equipo físico si se explican sus límites de aislamiento y capacidad.

Aspecto	Desarrollo	Producción de laboratorio
Topología	Un nodo como mínimo	Un control plane y dos workers como mínimo
Aplicación	Una réplica inicial por capa	Dos réplicas mínimas en frontend y backend
Escalado	Manual para experimentación	HPA en backend con rango inicial de 2 a 6
Publicación	HTTPS y nombre propio local	HTTPS y nombre distinto de desarrollo
Datos	Datos sintéticos modificables	Semilla controlada y persistencia independiente
Seguridad	Controles básicos activos	Políticas de red, RBAC y endurecimiento completos
Cambios	Construcción y pruebas frecuentes	Promoción de imagen aprobada y rollback
Observabilidad	Diagnóstico y métricas básicas	Paneles, alertas, logs y métricas de ambas BD

Planeación de capacidad

Elaborar inventario de CPU, RAM, disco, arquitectura de procesador, sistema operativo e IP por máquina. Como punto inicial de estimación, considerar 2 vCPU y 4 GiB para desarrollo, 2 vCPU y 4 GiB para el control plane y 4 vCPU y 8 GiB por worker de producción. No son garantías: medir el consumo de aplicación, bases, CNI y observabilidad y ajustar el presupuesto.

Un clúster multinodo de contenedores sobre un solo host permite practicar programación de Pods, pero no demuestra tolerancia a la pérdida de ese host.

Disponibilidad y límites

Distribuir las réplicas entre workers mediante restricciones de distribución o antiafinidad. Identificar como puntos únicos de fallo el control plane único, el balanceador único y cualquier base o volumen

sin redundancia. El HPA crea Pods, no máquinas. Documentar capacidad disponible y comportamiento cuando ya no caben más réplicas.

4 Arquitecturas y diagramas obligatorios

Arquitectura de la aplicación

Representar actores, interfaz, API, módulos de negocio, autenticación, base relacional y no relacional. Rotular protocolos y responsabilidades. Añadir un diagrama de secuencia de un CRUD y otro del flujo que usa ambos motores, incluyendo errores y recuperación.

Arquitectura de implementación

Crear un diagrama por ambiente con máquinas, roles, namespaces, Deployments, StatefulSets u operadores, Services, réplicas, entrada HTTPS, PVC, StorageClass y componentes de observabilidad. Mostrar dónde corre cada componente y qué dependencia tiene del almacenamiento.

Arquitectura de comunicación

Mostrar red física o virtual, gateway, firewall, red de administración, direcciones de nodos, CIDR de Pods y Services, DNS, rangos del balanceador y salida a Internet. Si hay VLAN, NAT o VPN, identificar sus interfaces y reglas. Evitar solapamiento de rangos con LAN y VPN.

Recorrido de referencia

Navegador → firewall y dirección de entrada → controlador de entrada con TLS → Service de frontend o API → réplicas de aplicación → Services internos de bases de datos. La ruta exacta dependerá de si la interfaz es SPA, renderizada en servidor o una combinación; justificarla.

Decisiones de arquitectura

- Registrar al menos seis decisiones: distribución Kubernetes, CNI, entrada y balanceador, persistencia, motores de datos y observabilidad. Añadir alternativa descartada, motivo, costo y limitación.
- Distinguir un Service que distribuye conexiones entre endpoints de un componente que publica tráfico fuera del clúster. Crear un recurso de entrada sin su controlador no publica la aplicación.
- Decidir entre Gateway API o Ingress con una implementación mantenida compatible con la versión elegida. Registrar versión, instalación y configuración; no depender de nombres de productos sin comprobar soporte.
- Especificar el mecanismo que proporciona una IP externa en laboratorio o nube. Un Service de tipo LoadBalancer necesita una implementación o integración que lo respalde.

Los diagramas deben coincidir con los manifiestos finales. Entregar tanto el archivo editable como la imagen incorporada al informe, con leyenda y nombres legibles.

5 Diseño de red y firewall

Antes de abrir puertos, construir una matriz de flujos. Usar puertos reales de la solución. Los valores siguientes son ejemplos de servicios habituales, no reglas universales para cualquier distribución.

Origen	Destino	Protocolo y puerto	Condición
Usuario	Entrada web	TCP 443	Permitido; 80 solo para redirección o desafío si se usa
Administrador	API Kubernetes	TCP 6443 o definido	Solo desde red de administración o VPN
Administrador	Nodos SSH	TCP 22 o definido	Solo administración autorizada
API	SGBD relacional	TCP 5432 o elegido	Solo Pods y usuarios de servicio autorizados
API	SGBD no relacional	TCP 27017 o elegido	Solo función autorizada
Pods	DNS del clúster	UDP y TCP 53	Permitir resolución interna
Monitor	Exporters y métricas	Puertos declarados	Solo componentes de observabilidad
Nodos	Nodos y control plane	Según CNI y distribución	Inventario oficial; no abrir indiscriminadamente

Controles obligatorios

- Aplicar firewall de host o perímetro y conservar su configuración exportable. Documentar orden de reglas, origen, destino, motivo y prueba asociada.
- Seleccionar un CNI que haga cumplir NetworkPolicy. Aplicar denegación inicial de entrada y salida y habilitar explícitamente DNS y los flujos necesarios, incluidos monitorización y respaldos.
- No publicar bases de datos mediante NodePort, IP pública o entrada web. Restringir interfaces de administración y paneles a operadores autorizados.
- Permitir entre nodos los puertos y protocolos necesarios para API, kubelet, almacenamiento y encapsulación del CNI seleccionado. Probar después de aplicar el firewall para evitar romper el clúster.

Validación de conectividad

Desde un Pod permitido, verificar conexión a cada base. Desde un Pod no autorizado, repetir contra el mismo destino y comprobar el bloqueo mientras el servicio continúa sano. Probar entrada y salida por

separado. Mostrar también que DNS funciona y que el acceso administrativo desde un origen no autorizado está restringido.

NetworkPolicy y firewall del host cumplen funciones distintas. Documentar qué protege cada control y cualquier tráfico fuera de su alcance, como componentes con hostNetwork.

6 Seguridad por capa

Capa	Implementación y evidencia requerida
Aplicación	Hash seguro de contraseñas, autorización de servidor, consultas parametrizadas u ORM, manejo seguro de errores y límites de intentos.
Navegador y API	TLS válido para el laboratorio, cookies seguras cuando apliquen, CSRF para autenticación por cookies y CORS con orígenes específicos si se necesita.
Datos	Usuarios de servicio con mínimo privilegio y sin rol administrativo. Separar cuenta de migración de cuenta de ejecución.
Contenedores	Usuario no root, sin privilegios, sin escalamiento de privilegios, capacidades reducidas y seccomp RuntimeDefault. Filesystem de solo lectura donde sea viable.
Kubernetes	RBAC por función, cuentas de servicio específicas, token desactivado donde no se requiere y políticas de admisión de seguridad.
Secretos	Credenciales fuera del repositorio y de imágenes. Procedimiento de creación, rotación y recuperación sin revelar valores.
Suministro	Versiones fijadas, imágenes trazables por digest o etiqueta inmutable y análisis de vulnerabilidades con tratamiento de hallazgos.
Infraestructura	Actualizaciones, acceso administrativo restringido, firewall, protección de kubeconfig y auditoría según capacidades.

Aplicar el perfil Restricted de Pod Security Admission a las cargas propias; documentar excepciones mínimas para componentes de plataforma que lo necesiten. No desactivar globalmente los controles para resolver un fallo de instalación.

Protección de información

Un Secret codificado en base64 no está cifrado por ese hecho. Documentar e implementar cifrado de secretos en reposo según la distribución, control de acceso y cifrado de respaldos. Configurar TLS hacia los motores de datos en producción de laboratorio y verificar la identidad del servidor con la CA correspondiente.

Pruebas y tratamiento de riesgos

Realizar pruebas únicamente sobre los recursos propios de la práctica. Comprobar denegación por rol, entradas inválidas, restricción de red, certificado y permisos de base. No incluir contraseñas, tokens ni llaves privadas en capturas. Registrar riesgos pendientes, impacto, mitigación y responsable; justificar cualquier excepción técnica.

7 Persistencia y gestión de datos

Diseño y despliegue

Desplegar ambos motores en cada ambiente con almacenamiento persistente y configuración declarativa. Puede emplearse StatefulSet u operador compatible. Definir StorageClass, tamaño, modo de acceso, política de recuperación, afinidad y comportamiento al mover la carga a otro nodo. Un StatefulSet no configura automáticamente replicación del motor.

- Registrar rutas lógicas, PVC y volúmenes de cada ambiente. Ningún ambiente compartirá credenciales o volúmenes de datos con el otro.
- Crear migraciones relacionales y scripts idempotentes para colecciones, índices, usuarios y datos sintéticos. No ejecutar migraciones concurrentemente en cada réplica de la aplicación.
- Si se usa almacenamiento local, explicar que recrear el Pod en otro nodo no garantiza acceso al volumen. La prueba de fallo de worker debe respetar esta limitación y documentar el tiempo de recuperación.

Respaldos y restauración obligatorios

Implementar respaldos programados de ambos motores, con método consistente según el SGBD, retención y destino separado del volumen de origen. Registrar cifrado, control de acceso, fecha, tamaño y verificación de integridad. No considerar una copia de archivos en caliente como respaldo consistente sin soporte del motor.

Restaurar cada base en una instancia o espacio de recuperación aislado, nunca sobre el origen de la prueba. Comprobar conteos, registros seleccionados, relaciones, índices y funcionamiento de la aplicación con los datos restaurados. Entregar comandos exactos y tiempos medidos.

Objetivos de recuperación

Definir antes de probar RPO y RTO: pérdida máxima aceptable de datos y tiempo máximo de recuperación. Como objetivo académico inicial pueden usarse RPO de 24 horas y RTO de 30 minutos; el alumno los ajustará y justificará. Reportar resultados reales, incluso si no cumple el objetivo.

Escalado y bases de datos

El escalado horizontal obligatorio corresponde a la capa de aplicación. No replicar una base incrementando replicas de forma arbitraria. Diseñar la alternativa de replicación y failover propia de cada motor, con consistencia, elección de primario y recuperación; implementarla será una ampliación si no forma parte del alcance aprobado.

8 Secuencia de implementación de los ambientes

El informe final debe convertir esta secuencia en un procedimiento ejecutable para la tecnología elegida. Cada paso incluirá prerequisito, archivo, comando exacto, resultado esperado, evidencia y solución de problemas. No basta escribir que se instaló o configuró un componente.

Paso 1 Preparar equipos

Inventariar recursos, fijar nombres e IP, configurar resolución, sincronización horaria y accesos. Verificar compatibilidad de CPU e imágenes. Documentar sistema operativo y versiones.

Paso 2 Crear los clústeres

Instalar la distribución elegida en desarrollo y producción con sus topologías. Guardar instrucciones o automatización y generar contextos claramente diferenciados. Verificar nodos Ready y componentes del sistema.

Paso 3 Habilitar plataforma

Instalar y validar CNI, DNS, almacenamiento, controlador de entrada y mecanismo de IP externa. Añadir Metrics Server o proveedor de métricas compatible. Fijar versiones de charts y valores utilizados.

Paso 4 Aplicar controles iniciales

Crear namespaces, cuotas, LimitRange, RBAC y políticas de seguridad. Crear secretos por procedimiento seguro. Aplicar NetworkPolicy con permisos explícitos para las dependencias.

Paso 5 Desplegar datos

Crear PVC, motores, usuarios y esquemas o colecciones. Verificar salud, credenciales, TLS, índices y semilla. Preparar Job único para migraciones y respaldos.

Paso 6 Publicar desarrollo

Construir imágenes, desplegar backend y frontend con sondas, recursos y configuración propia. Activar HTTPS. Ejecutar pruebas funcionales y registrar versión candidata.

Paso 7 Promover a producción

Usar el mismo artefacto probado, cambiar únicamente configuración del ambiente y aplicar réplicas, distribución, HPA y políticas de operación. Verificar datos independientes y ruta HTTPS.

Paso 8 Validar y reconstruir

Completar carga, fallos, seguridad y restauración. Repetir instalación en recursos limpios con el manual y registrar diferencias o pasos faltantes.

9 Escalado horizontal y balanceo de carga

Configuración mínima de producción

- Frontend y backend con al menos dos réplicas, Services estables y sondas startup, readiness y liveness acordes con la aplicación. La liveness no debe provocar reinicios masivos por una caída temporal de base de datos.
- Backend sin estado local de sesión ni archivos que deban compartirse entre réplicas. Resolver sesiones y archivos mediante mecanismos externos compatibles con el escalado.
- Definir requests y limits medidos. Configurar HPA autoscaling/v2 sobre backend con minReplicas 2, maxReplicas 6 y objetivo inicial de CPU del 60 por ciento, ajustable con justificación.
- Verificar API de métricas antes de probar. El porcentaje de CPU del HPA depende de requests; una métrica ausente no demuestra escalado funcional.
- Configurar distribución entre nodos, actualización gradual y PodDisruptionBudget coherente con las réplicas. Explicar que este último regula disrupciones voluntarias y no evita fallos imprevistos.

Experimento reproducible

1. Registrar línea base durante cinco minutos: réplicas, CPU, memoria, latencia p95, solicitudes por segundo y errores. Elegir una transacción representativa que consulte datos y evitar que el ensayo mida solo archivos estáticos.

2. Generar carga desde un equipo o Pod identificado, con herramienta, versión, script, concurrencia y duración registradas. Aumentar progresivamente durante al menos diez minutos o lo necesario para observar el HPA. Separar el consumo del generador de la aplicación.

3. Registrar serie temporal de réplicas deseadas, actuales y Ready. Observar crecimiento por encima de dos réplicas si la carga y capacidad lo permiten. Si no sucede, diagnosticar el cuello de botella y repetir con cambios justificados.

4. Reducir carga y esperar la ventana de estabilización declarada. Registrar retorno hacia el mínimo. Comparar métricas con y sin HPA usando el mismo escenario y datos.

Demostración del balanceo

Usar un identificador temporal de réplica en respuesta de diagnóstico o correlacionar peticiones con logs. Mostrar que varias réplicas atienden solicitudes; no exigir una distribución exactamente igual por conexión. Retirar la identificación pública al concluir la prueba.

Acordar antes de medir un umbral de latencia y errores, por ejemplo p95 menor de 1 segundo y errores menores de 1 por ciento bajo una carga definida. Reportar cumplimiento, saturación de BD y límites de conexiones; no prometer mejora lineal al añadir Pods.

10 Observabilidad operación y recuperación

Cobertura mínima

Objeto	Indicadores y evidencia
Nodos	Estado, CPU, memoria, disco, red y capacidad disponible.
Kubernetes	Pods no listos, reinicios, Pending, eventos y réplicas deseadas frente a disponibles.
Aplicación	Tráfico, errores, duración p95 y salud de dependencias; logs con fecha y correlación.
Base relacional	Disponibilidad, conexiones, consultas lentas o duración, capacidad y métricas del motor.
Base no relacional	Disponibilidad, conexiones, operaciones, latencia y capacidad.
Red y seguridad	Salud de entrada, expiración de TLS, rechazos observables y cambios relevantes.
Respaldos	Última ejecución, duración, tamaño, resultado y antigüedad del último respaldo válido.

Implementar una pila de métricas y paneles, por ejemplo Prometheus y Grafana o equivalente, más recolección centralizada de logs. Incluir exporters o integración para ambos motores con credenciales de monitorización de mínimo privilegio. Metrics Server no sustituye el historial y los paneles de observabilidad.

Alertas operativas

Configurar al menos cinco alertas: aplicación indisponible, nodo no disponible, tasa elevada de errores, saturación de recursos y respaldo fallido o vencido. Para cada una definir consulta, umbral, duración, severidad y acción. Provocar de manera controlada al menos dos y capturar activación y resolución sin enviar datos a terceros.

Pruebas de fallos

- Eliminar un Pod de backend y registrar recreación, balanceo y efecto en solicitudes.
- Realizar mantenimiento voluntario de un worker con drain y luego uncordon, comprobando presupuesto de disrupción y capacidad del otro nodo. No usar opciones destructivas para forzar la prueba.
- Simular indisponibilidad temporal de una base y comprobar manejo de errores y recuperación. Registrar pérdida o retraso de operaciones y reconciliación entre motores.
- Demostrar actualización gradual a una versión distinta y rollback. Las migraciones deberán ser compatibles o tener un plan explícito de reversión; rollback de imagen no revierte automáticamente el esquema.

Registrar línea de tiempo y afectación real. No afirmar alta disponibilidad por mostrar únicamente dos Pods en estado Running. Añadir un manual de incidentes con diagnóstico, acciones, verificación y escalamiento para cada escenario.

11 Repositorio automatización y documentos

Organización mínima del repositorio

Mantener carpetas identificables para frontend, backend, pruebas, infraestructura base, configuración de desarrollo, configuración de producción, observabilidad, red, respaldos y documentación. Usar Kustomize, Helm u otro mecanismo declarativo para evitar divergencias manuales entre ambientes.

El repositorio debe incluir Dockerfiles, archivo de dependencias fijadas, manifiestos o charts, valores sin secretos, scripts, migraciones, semillas, pruebas y README de inicio. Excluir claves, archivos kubeconfig privilegiados y dumps con información sensible.

Expediente técnico obligatorio

- D01 Ficha del proyecto y requerimientos funcionales y no funcionales con criterios de aceptación.
- D02 Investigación de Kubernetes, comparación de tecnologías y decisiones de arquitectura.
- D03 Diagramas de aplicación, despliegue de ambos ambientes y red; inventario y presupuesto de capacidad.
- D04 Modelos y diccionarios de ambas bases, migraciones, consistencia y reglas de negocio.
- D05 Manual de instalación desde cero con versiones, dependencias, orden, comandos y verificación de cada ambiente.
- D06 Plan y configuración de seguridad con matriz RBAC, flujos, firewall, TLS, secretos y riesgos residuales.
- D07 Manual de publicación, promoción, actualización, rollback y administración de accesos.
- D08 Manual de operación, observabilidad, alertas y solución de incidentes.
- D09 Plan de respaldos, restauración y continuidad con RPO y RTO medidos.
- D10 Plan de pruebas, resultados, evidencias, conclusiones, limitaciones y referencias.

Integración y entrega

Estos documentos pueden formar capítulos de un informe único, siempre que estén identificados y vinculados con los archivos del repositorio. Entregar informe en PDF y formato editable, diagramas editables y una versión etiquetada del código. Automatizar construcción, pruebas y validación de manifiestos; documentar promoción controlada de la imagen aprobada a producción.

12 Evidencias y matriz de aceptación

Cada evidencia tendrá identificador, fecha, ambiente, requisito, comando o acción, resultado esperado, resultado observado y explicación. Combinar capturas legibles con salidas de texto, scripts y métricas exportadas. Una captura aislada de un panel no acredita una prueba completa.

ID	Prueba y condición de aceptación
E01	Dos contextos y topologías distintas; nodos y componentes sanos.
E02	URLs HTTPS distintas y datos independientes entre ambientes.
E03	CRUD de tres entidades, roles y validaciones verificadas desde interfaz y API.
E04	Uso funcional y persistente de ambas bases con consultas trazables.
E05	Recreación de Pods de datos sin pérdida de registros en condiciones documentadas.
E06	HPA crece por encima del mínimo y luego disminuye; series temporales y carga adjuntas.
E07	Solicitudes atendidas por varias réplicas y réplicas distribuidas entre workers.
E08	Conexiones permitidas y denegadas por políticas; DNS y servicios sanos como control.
E09	Acceso denegado por rol de aplicación, RBAC y usuario de base con permisos limitados.
E10	Paneles con nodos, servicios y ambos motores; dos alertas activadas y resueltas.
E11	Fallo de Pod y mantenimiento de worker con tiempos e impacto medidos.
E12	Actualización y rollback con comprobación de versión y funcionalidad.
E13	Respaldo y restauración verificada de ambas bases; RPO y RTO registrados.
E14	Reconstrucción desde recursos limpios usando manual, sin pasos tácitos ni secretos en Git.

Bitácora de pruebas

Usar un registro por prueba con: requisito, precondiciones, datos, pasos, parámetros, duración, mediciones, resultado y vínculo a evidencia. Cuando una prueba falle, conservar el fallo, describir diagnóstico y añadir la repetición; no sustituir mediciones por estimaciones.

El docente podrá solicitar cambios pequeños o ejecutar una prueba no anunciada dentro del alcance para verificar autoría y comprensión. El alumno debe poder localizar y explicar los recursos que controlan cada comportamiento.

13 Calendario y evaluación

Semana	Resultado esperado
1	Proyecto aprobado, investigación, requerimientos y arquitecturas preliminares.
2	Aplicación multicapa con modelos y CRUD inicial en ambos motores.
3	Contenedores, clúster de desarrollo y publicación HTTPS funcional.
4	Clúster productivo, red, almacenamiento y separación de configuración.
5	Seguridad por capas, secretos, firewall y validación de aislamiento.
6	Métricas, logs, paneles, alertas y respaldos.
7	HPA, carga, fallos, restauración, actualización y rollback.
8	Reconstrucción, informe final, entrega etiquetada y defensa individual.

Rúbrica de evaluación sobre 100 puntos

Criterio	Puntos
Proyecto original y aplicación multicapa funcional	15
Arquitecturas de aplicación despliegue y red justificadas	10
Dos ambientes reproducibles y configuración declarativa	15
Uso de ambas bases persistencia y restauración	10
Balanceo HPA y pruebas de carga medibles	15
Seguridad de aplicación datos red e infraestructura	15

Criterio	Puntos
Observabilidad alertas y respuesta a fallos	10
Documentación evidencias y defensa individual	10

En cada criterio: completo y demostrado obtiene 100 por ciento del puntaje; funcional con omisiones menores, 75 por ciento; parcial o con evidencia insuficiente, 50 por ciento; explicación sin implementación verificable, hasta 25 por ciento; ausente, cero. El docente registrará la justificación del puntaje.

La entrega se considerará completa cuando exista aplicación CRUD funcional, dos clústeres, uso de ambos motores, escalado demostrado, pruebas de seguridad y restauración. Si falta un requisito esencial se registrará como pendiente de corrección, aunque se haya evaluado el trabajo parcial. No se aceptan evidencias ajenas ni resultados fabricados.

14 Estructura del informe y fuentes de consulta

Guion del documento final del estudiante

Portada y control de versión; resumen del proyecto; requerimientos; investigación; decisiones; diagramas; inventario; modelos de datos; preparación de equipos; instalación paso a paso de desarrollo; instalación paso a paso de producción; seguridad; publicación; operación; pruebas; recuperación; resultados; limitaciones; conclusiones; referencias y anexos.

Plantilla de un paso del manual

Número y objetivo del paso. Ambiente y contexto. Requisitos previos. Ruta y versión del archivo. Comando exacto ejecutado sin secretos. Explicación de sus parámetros. Resultado esperado. Evidencia identificada. Comprobación de éxito. Error posible y diagnóstico. Procedimiento de reversión cuando corresponda.

Comprobaciones de diagnóstico que debe adaptar

```
kubectl config get-contexts
kubectl --context <contexto> get nodes -o wide
kubectl --context <contexto> -n <namespace> get pods,svc,pvc,hpa
kubectl --context <contexto> -n <namespace> describe hpa <nombre>
kubectl --context <contexto> -n <namespace> top pods
kubectl --context <contexto> -n <namespace> get networkpolicy
kubectl --context <contexto> auth can-i <verbo> <recurso> -n <namespace>
```

Los marcadores entre signos angulares se sustituyen por valores propios. Estos comandos son controles de diagnóstico y no constituyen por sí solos el manual de instalación. Las comprobaciones de permisos deben ejecutarse con la identidad evaluada.

Fuentes oficiales para la investigación

Kubernetes Production environment — https://kubernetes.io/docs/setup/production-environment/

Kubernetes Horizontal Pod Autoscaling — https://kubernetes.io/docs/concepts/workloads/autoscaling/horizontal-pod-autoscale/

Kubernetes Network Policies — https://kubernetes.io/docs/concepts/services-networking/network-policies/

Kubernetes Security Checklist — https://kubernetes.io/docs/concepts/security/security-checklist/

Kubernetes Application Security Checklist — https://kubernetes.io/docs/concepts/security/application-security-checklist/

Consulta de referencia del diseño de esta práctica: 16 de octubre de 2026. El estudiante añadirá documentación oficial de la distribución, CNI, controlador de entrada, almacenamiento, motores y herramientas elegidas, con versión y fecha de consulta. No debe asumir que una guía de otra versión se aplica sin cambios.

15 Glosario de Kubernetes y despliegue

Este glosario apoya la investigación, la implementación y la defensa de la práctica. Las definiciones deben relacionarse con los recursos y decisiones del proyecto personal.

Kubernetes. Plataforma que coordina aplicaciones en contenedores mediante una configuración declarativa y mantiene su estado deseado.

Clúster. Conjunto de componentes de control y nodos donde se ejecutan las cargas. En esta práctica se implementa uno para desarrollo y otro para producción.

Plano de control o control plane. Componentes que administran el estado del clúster, reciben solicitudes, programan Pods y ejecutan controladores.

Nodo y worker. Un nodo es una máquina física o virtual del clúster. Un worker se dedica a ejecutar cargas de trabajo.

API server. Punto de entrada a la API de Kubernetes. Valida solicitudes y aplica los mecanismos de autenticación y autorización configurados.

etcd. Almacén distribuido de clave y valor utilizado por Kubernetes para guardar el estado del clúster; requiere protección y respaldo propios.

Scheduler. Componente que selecciona un nodo para un Pod pendiente según recursos disponibles, restricciones y políticas.

Kubelet. Agente del nodo que coordina con el runtime la ejecución de los contenedores definidos para los Pods asignados.

Imagen y contenedor. La imagen empaqueta aplicación y dependencias. Un contenedor es una instancia de ejecución creada a partir de una imagen.

Pod. Unidad mínima de despliegue de Kubernetes. Agrupa uno o varios contenedores que comparten red y pueden compartir volúmenes.

Deployment y réplica. Deployment administra Pods intercambiables y sus actualizaciones. Una réplica es una instancia de la carga que se desea mantener disponible.

StatefulSet. Recurso que proporciona identidad estable y gestión ordenada de Pods con estado. Por sí mismo no implementa replicación ni alta disponibilidad de una base de datos.

Job y CronJob. Job ejecuta una tarea hasta su finalización; CronJob crea tareas según un calendario. Se pueden usar para migraciones o respaldos.

Namespace. Ámbito lógico para organizar recursos dentro de un clúster. No equivale a un clúster independiente ni garantiza aislamiento por sí solo.

Manifiesto y estado deseado. Un manifiesto describe recursos, normalmente en YAML. El estado deseado es la configuración que los controladores intentan mantener.

16 Glosario de configuración y escalado

kubectl y contexto. kubectl es la herramienta de línea de comandos para la API. Un contexto identifica la combinación de clúster, usuario y namespace utilizada.

Kubeconfig. Archivo que configura acceso a clústeres y contextos; puede contener credenciales o referencias a ellas y debe protegerse.

Helm y chart. Helm administra paquetes de recursos Kubernetes. Un chart contiene plantillas y valores para instalar y configurar una solución.

Kustomize y overlay. Kustomize adapta manifiestos sin plantillas. Un overlay aplica cambios de un ambiente sobre una base común.

ConfigMap y Secret. ConfigMap guarda configuración no confidencial. Secret representa datos sensibles; la codificación base64 no constituye cifrado.

Requests y limits. Requests declara recursos utilizados para programar la carga y como referencia de algunas métricas. Limits fija límites de uso; exceder memoria puede causar terminación y la CPU puede ser restringida.

ResourceQuota y LimitRange. ResourceQuota limita consumo o cantidad de recursos por namespace. LimitRange establece valores por defecto y restricciones para solicitudes y límites.

HPA. Horizontal Pod Autoscaler ajusta el número de réplicas de una carga según métricas y objetivos. No crea nodos ni garantiza capacidad física disponible.

Escalado horizontal y vertical. El horizontal aumenta o reduce instancias; el vertical cambia los recursos asignados a cada instancia.

Metrics Server. Componente que recopila métricas de recursos y las expone mediante la API de métricas. No sustituye una plataforma de observabilidad con historial.

Readiness probe. Comprobación que determina si un contenedor está listo para atender tráfico. Un Pod no listo normalmente se retira de los destinos disponibles del Service.

Liveness y startup probe. Liveness detecta una condición que requiere reiniciar el contenedor. Startup permite completar un arranque lento antes de ejecutar las otras comprobaciones.

Antiafinidad y distribución topológica. Reglas para separar o distribuir Pods entre nodos o dominios de fallo. Su eficacia depende de restricciones y capacidad disponibles.

PodDisruptionBudget o PDB. Presupuesto que limita disrupciones voluntarias mediante el mecanismo de desalojo. No evita fallos de hardware ni todas las formas de eliminación de Pods.

Rolling update y rollback. Rolling update sustituye réplicas gradualmente. Rollback recupera una revisión anterior de la carga; no revierte automáticamente datos ni migraciones.

Drain y uncordon. Drain prepara un nodo para mantenimiento desalojando cargas según las condiciones aplicables. Uncordon permite que vuelva a recibir nuevos Pods.

17 Glosario de redes y comunicación

Service y endpoint. Service ofrece acceso estable a un conjunto de destinos. Un endpoint identifica un destino de red; Kubernetes utiliza EndpointSlices para representar estos destinos.

ClusterIP y NodePort. ClusterIP publica un Service mediante una IP interna del clúster. NodePort lo hace accesible en un puerto de los nodos, sujeto a red y firewall.

Service LoadBalancer. Tipo de Service que solicita un balanceador externo. Requiere una implementación compatible en nube o laboratorio.

Balanceo de carga. Distribución de tráfico entre destinos disponibles. No implica necesariamente que cada réplica reciba exactamente la misma cantidad de solicitudes.

Ingress y controlador de entrada. Ingress define reglas HTTP y HTTPS. Su controlador interpreta las reglas y atiende el tráfico; crear solo el recurso no basta.

Gateway API. Conjunto de recursos para modelar entrada y encaminamiento de tráfico con roles diferenciados. Requiere una implementación compatible.

CNI. Container Network Interface es una especificación de integración de red para contenedores. La implementación elegida debe soportar las funciones requeridas, incluidas las políticas de red.

NetworkPolicy. Recurso que declara comunicaciones permitidas para Pods según selectores y reglas. Requiere soporte de cumplimiento por el componente de red.

Default deny. Estrategia que bloquea inicialmente el tráfico seleccionado y permite solo flujos explícitos. Debe considerar entrada, salida, DNS y dependencias operativas.

Ingress y egress de red. En el contexto de políticas, ingress significa tráfico entrante y egress saliente. No confundir ingress de tráfico con el recurso Ingress.

DNS. Sistema que resuelve nombres a direcciones. En el clúster permite localizar servicios por nombre sin depender de IP de Pods cambiantes.

CIDR. Notación de prefijo que expresa un rango de direcciones IP, por ejemplo 10.20.0.0/16. Los rangos de nodos, Pods y Services deben planearse sin conflictos.

Firewall. Control que permite o bloquea comunicaciones según reglas. Puede actuar en host, perímetro o plataforma y complementa las políticas entre Pods.

NAT y VLAN. NAT traduce direcciones de red. VLAN divide una red de nivel de enlace en segmentos lógicos; la comunicación entre segmentos necesita encaminamiento y controles.

VPN y red de administración. VPN establece conectividad protegida entre redes o equipos. La red de administración restringe el acceso a interfaces operativas.

TLS y HTTPS. TLS protege comunicaciones mediante cifrado e identificación del servidor con certificados. HTTPS es HTTP sobre TLS; debe verificarse la confianza y el nombre del certificado.

18 Glosario de seguridad y datos

Autenticación y autorización. Autenticación verifica la identidad; autorización determina qué acciones puede realizar esa identidad.

RBAC y mínimo privilegio. RBAC asigna permisos mediante roles y vínculos. Mínimo privilegio concede únicamente los accesos necesarios para una función.

ServiceAccount. Identidad de una carga dentro de Kubernetes. No debe confundirse con una cuenta de usuario de la aplicación o de la base de datos.

Pod Security Admission y Restricted. Mecanismo de admisión que aplica estándares de seguridad a Pods. Restricted es el perfil más exigente de los estándares incorporados.

SecurityContext y seccomp. SecurityContext define opciones de seguridad de Pods o contenedores. Seccomp limita llamadas al sistema; RuntimeDefault aplica el perfil predeterminado del runtime.

Hash y cifrado. Un hash produce un resumen y no se descifra; las contraseñas requieren funciones de hash adecuadas con sal. El cifrado permite recuperar información usando una clave.

CSRF y CORS. CSRF es una falsificación de solicitudes desde otro sitio usando la sesión de la víctima. CORS regula lecturas entre orígenes en navegadores y no sustituye autenticación o autorización.

CRUD y API. CRUD comprende crear, consultar, actualizar y eliminar registros. Una API expone operaciones y contratos para que otros componentes interactúen con la aplicación.

Base relacional y no relacional. Una base relacional organiza datos mediante relaciones y restricciones. No relacional agrupa modelos como documentos, clave y valor o grafos; su consistencia depende del motor y la configuración.

Transacción e idempotencia. Una transacción agrupa operaciones con garantías definidas por el motor. Una operación idempotente puede repetirse sin producir efectos adicionales no deseados.

Outbox y fuente de verdad. Outbox registra un evento pendiente junto con el cambio principal en una misma transacción para enviarlo después. La fuente de verdad es el sistema considerado autoritativo para un dato.

Migración y semilla. Una migración versiona cambios de esquema o estructura. Una semilla crea datos iniciales o de prueba de manera controlada.

PV y PVC. PersistentVolume representa almacenamiento disponible. PersistentVolumeClaim solicita almacenamiento con características específicas para una carga.

StorageClass y CSI. StorageClass describe clases y parámetros de aprovisionamiento. Container Storage Interface permite integrar controladores de almacenamiento.

Persistencia y replicación. Persistencia conserva datos más allá de la vida de un contenedor. Replicación mantiene copias coordinadas; ninguna sustituye por sí sola un respaldo verificado.

19 Glosario de observabilidad y continuidad

Observabilidad. Capacidad para comprender el estado interno de un sistema mediante señales como métricas, logs y trazas.

Métrica y serie temporal. Una métrica es una medición numérica. Una serie temporal registra sus valores a lo largo del tiempo con sus etiquetas de identificación.

Log y correlación. Un log registra eventos. Un identificador de correlación relaciona mensajes de una misma solicitud entre componentes.

Exporter y dashboard. Un exporter expone métricas de un sistema para su recopilación. Un dashboard presenta indicadores que permiten interpretar su comportamiento.

Alerta y umbral. Una alerta notifica una condición definida mediante una regla. El umbral y su duración deben distinguir problemas sostenidos de fluctuaciones normales.

Latencia p95. Valor de latencia que no supera aproximadamente el 95 por ciento de las observaciones del periodo. Debe acompañarse de la carga y ventana de medición.

Throughput y tasa de errores. Throughput expresa operaciones completadas por unidad de tiempo. La tasa de errores mide la proporción o frecuencia de operaciones fallidas.

Prueba de carga y cuello de botella. La prueba de carga mide respuesta bajo demanda controlada. Un cuello de botella es el recurso o dependencia que limita el rendimiento del conjunto.

SLO. Objetivo de nivel de servicio expresado con una métrica, un valor y un periodo, como una proporción de solicitudes que cumple una latencia objetivo.

Alta disponibilidad y dominio de fallo. Alta disponibilidad busca mantener el servicio ante fallos mediante redundancia y recuperación. Un dominio de fallo agrupa recursos que pueden perderse por una misma causa.

Failover. Transferencia de servicio a una instancia alternativa tras una falla. Su implementación debe considerar detección, estado, consistencia y prevención de múltiples primarios.

Respaldo y restauración. Un respaldo es una copia recuperable tomada mediante un método consistente. Restauración reconstruye datos desde esa copia y debe probarse, no suponerse.

RPO. Recovery Point Objective es la pérdida máxima de datos tolerable expresada en tiempo. Se compara con el punto de recuperación realmente disponible.

RTO. Recovery Time Objective es el tiempo objetivo máximo para recuperar el servicio. La práctica debe medirlo y declarar desde qué evento se cuenta.

CI y CD. Integración continua automatiza integración y validación de cambios. Entrega o despliegue continuo automatiza su preparación o publicación según la política definida.

Digest y trazabilidad. El digest identifica el contenido de una imagen. La trazabilidad vincula código, imagen, configuración, versión desplegada y evidencia de validación.